Artificial Intelligence

BS (CS) _SP_2026

Lab_01 Tasks

Learning Objectives:

1. Python Basic
2. Operators
3. Variables
4. Data Type
5. Control Statements
6. List, Sets, Tuples, Dictionaries
7. Classes and Objects
8. Local and Global Variables
9. Oop Concepts (Polymorphism, Abstraction, Inheritance, Encapsulation)

Lab Tasks

Submission Instructions

1. Create a separate file for each question with name *ROLLNO_SEC_LAB01* **e.g. i23XXXX_A_LAB01**
2. Compress all your **.py** files along with output **screenshots** directly into a **.zip file**.
3. Now you have to submit this zipped file on Google Classroom under your section heading.
4. If you don't follow the above-mentioned submission instruction, you will be marked **zero.**

Plagiarism in the Lab Task will result in **zero** marks in the whole category.

Q1. Write a Python program for a sports league that manages a tuple of player information. Implement control statements to sort the players based on their performance metrics, such as goals scored, assists, and playing time.

Q2. Write a python code that perform following tasks:

data_points = [(2, 5), (1, 3), (4, 8), (3, 6), (2, 7)]

a. Sort the tuples based on the sum of their elements in descending order.
b. Filter the tuples to include only those where both elements are not a prime numbers.
c. Create a new tuple by extracting the second element from each tuple.
d. Calculate the product of all elements in the new tuple.

Q3. Write a python program for an online shopping system using class inheritance. Design a base class `**Product**` with attributes like `**product_id**`, `**name**`, and `**price**`. Implement methods, including `**display_details()**`, to showcase basic product information. Derive two classes, `**Electronics**` and `**Clothing**`, inheriting from `**Product** inheriting and extending attributes like **brand** for electronics and **size** for clothing. Customize the `**display_details()**` method for additional information in each derived class. These classes should also include an `**apply_discount()**` method for. products on sale. To show shopping operations, develop a function, `**shopping_menu(cart)**`, that takes a list of product instances (mix of electronics and clothing) and showcases operations like adding items to the shopping cart, calling the **display_details()** method for each product, applying discounts, and displaying the total price.

Q4. Create a class representing a library book. Each book should have attributes such as book ID, title, author, and availability status. Implement methods to check out a book avaliability, return a book, and display the book details. Use a list or set to manage the collection of books. Implement control statements to handle book transactions and availability.

Q5. Create a Python class for a rectangle that can calculate and return both the area and diagonal of the rectangle.

$$Area = length * width$$

$$Diagonal = \sqrt{(length^2 + width^2)}$$

• Create an object of the Rectangle class with specific length and width • Call the area()and diagonal() methods, and print the results.

Input Example:

Length = 5
Width = 3

Expected Output:

Length: 5

Width: 3

Area: 15

Diagonal: 5.83

Q6. Write a Python program for a library management system using class inheritance. Design a base class **'LibraryItem'** with attributes like '**item_id**','**title**', and **'availability'**. Implement methods, including **'check_out()'** and **'return_item()'**, to manage borrowing and returning items. Derive two classes, **'Book'** and **'ResearchThesis'**, inheriting from **'LibraryItem'**. Include specific attributes (e.g.,**'author'** for books, **'Thesis_Topic'** for ResearchThesis) and customize the 'get_info()' method in each derived class for additional information. Implement methods such as 'calculate_penalty()' for late returns. To simulate library operations, create a function, 'simulate_library_operations(library_items)', that takes a list of item instances (mix of books and ResearchThesis) and demonstrates operations like checking out and returning items, displaying information, and calculating penalties for late returns.

Note: Ensure to focus on the class inheritance, method overriding, and polymorphic behavior while designing this library management system.